

# Module 3

## Problem Statement 1:- Merge Sort - Placement Packages

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void merge(vector<int> &arr, int low, int mid, int high)
6  {
7      vector<int> temp;
8
9      int i = low;
10     int j = mid + 1;
11
12     while(i <= mid && j <= high)
13     {
14         if(arr[i] <= arr[j])
15         {
16             temp.push_back(arr[i]);
17             i++;
18         }
19         else
20         {
21             temp.push_back(arr[j]);
22             j++;
23         }
24     }
25
26     while(i <= mid)
27     {
28         temp.push_back(arr[i]);
29         i++;
30     }
31
32     while(j <= high)
33     {
34         temp.push_back(arr[j]);
35         j++;
36     }
37
38     for(int k = low; k <= high; k++)
39     {
40         arr[k] = temp[k - low];
41     }
42 }
43
44 void mergeSort(vector<int> &arr, int low, int high)
45 {
46     if(low >= high)
47         return;
48
49     int mid = (low + high) / 2;
50
51     mergeSort(arr, low, mid);
52
53     mergeSort(arr, mid + 1, high);
54
55     merge(arr, low, mid, high);
56 }
57
58 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
59 ● PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps1.cpp
60 ● PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
61 5
62 4 2 5 1 3
63 1 2 3 4 5
64 ❖ PS C:\Users\manoj\OneDrive\Documents\DSA C++> |
```

## Problem Statement 2:- Merge Sort - Order Processing Analysis

```
5 void merge(vector<int> &arr, int low, int mid, int high)
38 for(int k = low; k <= high; k++)
42 }
43
44 void mergeSort(vector<int> &arr, int low, int high)
45 {
46     if(low >= high)
47         return;
48
49     int mid = (low + high) / 2;
50
51     mergeSort(arr, low, mid);
52     mergeSort(arr, mid + 1, high);
53
54     merge(arr, low, mid, high);
55 }
56
57 int main()
58 {
59     int n;
60     cin >> n;
61
62     vector<int> arr(n);
63
64     for(int i = 0; i < n; i++)
65         cin >> arr[i];
66
67     mergeSort(arr, 0, n - 1);
68
69     cout << "Sorted Array : ";
70
71     for(int i = 0; i < n; i++)
72         cout << arr[i] << " ";
73
74     cout << endl;
75
76     int median = arr[n / 2];
77
78     cout << "Median : " << median << endl;
79
80     int count = 0;
81
82     for(int i = 0; i < n; i++)
83     {
84         if(arr[i] > median)
85             count++;
86     }
87
88     cout << "Orders Above Median : " << count << endl;
89
90     int diff = arr[n - 1] - arr[0];
91
92     cout << "Difference : " << diff;
93
94     return 0;
95 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
● PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps2.cpp
● PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
8
10 7 8 9 1 5 3 2
Sorted Array : 1 2 3 5 7 8 9 10
Median : 7
Orders Above Median : 3
Difference : 9
```

### Problem Statement 3:- Quick Sort - Server Response Time

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int partition(vector<int> &arr, int low, int high)
6  {
7      int pivot = arr[high];
8      int i = low - 1;
9
10     for(int j = low; j < high; j++)
11     {
12         if(arr[j] < pivot)
13         {
14             i++;
15             swap(arr[i], arr[j]);
16         }
17     }
18
19     swap(arr[i + 1], arr[high]);
20
21     return i + 1;
22 }
23
24 void quickSort(vector<int> &arr, int low, int high)
25 {
26     if(low < high)
27     {
28         int pivotIndex = partition(arr, low, high);
29
30         quickSort(arr, low, pivotIndex - 1);
31
32         quickSort(arr, pivotIndex + 1, high);
33     }
34 }
35
36 int main()
37 {
38     int n;
39     cin >> n;
40
41     vector<int> arr(n);
42
43     for(int i = 0; i < n; i++)
44         cin >> arr[i];
45
46     quickSort(arr, 0, n - 1);
47
48     for(int i = 0; i < n; i++)
49         cout << arr[i] << " ";
50
51     return 0;
52 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

● PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps3.cpp

● PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

6  
12 11 13 5 6 7  
5 6 7 11 12 13

## Problem Statement 4:- Quick Sort - Trade Value Analysis

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int partition(vector<long long> &arr, int low, int high)
6  {
7      long long pivot = arr[high];
8      int i = low - 1;
9
10     for(int j = low; j < high; j++)
11     {
12         if(arr[j] > pivot)
13         {
14             i++;
15             swap(arr[i], arr[j]);
16         }
17     }
18
19     swap(arr[i + 1], arr[high]);
20
21     return i + 1;
22 }
23
24 void quickSort(vector<long long> &arr, int low, int high)
25 {
26     if(low < high)
27     {
28         int pivotIndex = partition(arr, low, high);
29
30         quickSort(arr, low, pivotIndex - 1);
31
32         quickSort(arr, pivotIndex + 1, high);
33     }
34 }
35
36 int main()
37 {
38     int n;
39     cin >> n;
40
41     vector<long long> arr(n);
42
43     for(int i = 0; i < n; i++)
44         cin >> arr[i];
45
46     quickSort(arr, 0, n-1);
47
48     cout<<"Sorted Array : ";
49
50     for(int i=0;i<n;i++)
51         cout<<arr[i]<<" ";
52
53     cout<<endl;
54
55     cout<<"Top 5 : ";
56
57     // ... (rest of the code for top 5 calculation) ...
58 }
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS GITLENS

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps4.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
10
45 78 12 89 56 90 34 67 23 99
Sorted Array : 99 90 89 78 67 56 45 34 23 12
Top 5 : 99 90 89 78 67
Average of Top 5 : 84.6
Values Above Overall Average : 5
```

## Problem Statement 5:- Heap Sort - Tournament Scores

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void heapify(vector<int> &arr, int n, int i)
6  {
7      int largest = i;
8
9      int left = 2 * i + 1;
10     int right = 2 * i + 2;
11
12     if(left < n && arr[left] > arr[largest])
13         largest = left;
14
15     if(right < n && arr[right] > arr[largest])
16         largest = right;
17
18     if(largest != i)
19     {
20         swap(arr[i], arr[largest]);
21
22         heapify(arr, n, largest);
23     }
24 }
25
26 void heapSort(vector<int> &arr, int n)
27 {
28     for(int i = n / 2 - 1; i >= 0; i--)
29         heapify(arr, n, i);
30
31     for(int i = n - 1; i > 0; i--)
32     {
33         swap(arr[0], arr[i]);
34
35         heapify(arr, i, 0);
36     }
37 }
38
39 int main()
40 {
41     int n;
42     cin >> n;
43
44     vector<int> arr(n);
45
46     for(int i = 0; i < n; i++)
47         cin >> arr[i];
48
49     heapSort(arr, n);
50
51     for(int i = 0; i < n; i++)
52         cout << arr[i] << " ";
53
54     return 0;
55 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

- PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps5.cpp
- PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe

8  
100 50 75 25 90 60 80 40  
25 40 50 60 75 80 90 100

## Problem Statement 6:- Heap Sort - Hospital Response Analysis

```
1  #include <iostream>
2  #include <vector>
3  #include <iomanip>
4  using namespace std;
5
6  void heapify(vector<int> &arr, int n, int i)
7  {
8      int largest = i;
9
10     int left = 2 * i + 1;
11     int right = 2 * i + 2;
12
13     if(left < n && arr[left] > arr[largest])
14         largest = left;
15
16     if(right < n && arr[right] > arr[largest])
17         largest = right;
18
19     if(largest != i)
20     {
21         swap(arr[i], arr[largest]);
22         heapify(arr, n, largest);
23     }
24 }
25
26 void heapSort(vector<int> &arr, int n)
27 {
28     for(int i = n/2 - 1; i >= 0; i--)
29         heapify(arr, n, i);
30
31     for(int i = n-1; i > 0; i--)
32     {
33         swap(arr[0], arr[i]);
34         heapify(arr, i, 0);
35     }
36 }
37
38 int main()
39 {
40     int n;
41     cin >> n;
42
43     vector<int> arr(n);
44
45     for(int i=0;i<n;i++)
46         cin>>arr[i];
47
48     heapSort(arr,n);
49
50     cout<<"Sorted Array : ";
51
52     for(int i=0;i<n;i++)
53         cout<<arr[i]<<" ";
54
55     cout<<endl;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
● PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
10
45 78 12 89 56 90 34 67 23 99
Sorted Array : 12 23 34 45 56 67 78 89 90 99
Fastest : 12
Slowest : 99
Average : 59.30
Cases Faster Than Average : 5
Percentage : 50.00%
```